

Unsorted Singly Linked List

Unsorted Singly Linked List

Introduction

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for the linked list
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Function to create a new node
Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}
```

Unsorted Singly Linked List

Operations: Overview

- ▶ **Search:** Linear search through the list is all that can be done.
- ▶ **Insert:** Insert at the head of the list.
- ▶ **Delete (specified with pointer):** Locate the previous node; make it point to the next.
- ▶ **Delete (specified with key):** Locate the node and the previous node. Make the previous node point to the next.
- ▶ **FindMin:** Linear search through the list is all that can be done.
- ▶ **DeleteMin:** Locate the node and the previous node. Make the previous node point to the next.
- ▶ **DecreaseKey:** Decrease the key of the pointed node.
- ▶ **Meld:** Locate the last node of the first list, make it point to the head of the other.

Unsorted Singly Linked List

Time bounds

If n is the current size of the data structure,

- ▶ **Search:** $O(n)$
- ▶ **Insert:** $O(1)$
- ▶ **Delete (specified with pointer):** $O(n)$.
- ▶ **Delete (specified with key):** $O(n)$.
- ▶ **FindMin:** $O(n)$.
- ▶ **DeleteMin:** $O(n)$.
- ▶ **DecreaseKey:** $O(1)$.
- ▶ **Meld:** $O(n)$. ($O(1)$ if a pointer to the last node is available.)

Unsorted Singly Linked List

Insert Operation

```
// Function to insert a node at the beginning of the list
void insert(Node** headRef, int value) {
    Node* newNode = createNode(value);
    newNode->next = *headRef;
    *headRef = newNode;
}
```

Unsorted Singly Linked List

Search Operation

```
// Function to search for a key in the list
Node* search(Node* head, int key) {
    Node* current = head;
    while (current != NULL) {
        if (current->data == key) {
            return current;
        }
        current = current->next;
    }
    return NULL;
}
```

Unsorted Singly Linked List

Delete by Pointer Operation

```
// Function to delete a node by pointer
void deleteByPtr(Node** headRef, Node* nodeToDelete) {
    if (*headRef == NULL || nodeToDelete == NULL) {
        return;
    }
    if (*headRef == nodeToDelete) {
        *headRef = (*headRef)->next;
        free(nodeToDelete);
        return;
    }
    Node* current = *headRef;
    while (current->next != nodeToDelete && current->next != NULL) {
        current = current->next;
    }
    if (current->next == NULL) {
        printf("Node not found\n");
        return;
    }
    current->next = nodeToDelete->next;
    free(nodeToDelete);
}
```

Unsorted Singly Linked List

Delete by Key Operation

```
// Function to delete a node by key
void deleteByKey(Node** headRef, int key) {
    Node* current = *headRef;
    Node* prev = NULL;
    while (current != NULL && current->data != key) {
        prev = current;
        current = current->next;
    }
    if (current == NULL) {
        printf("Key not found\n");
        return;
    }
    if (prev == NULL) {
        *headRef = current->next;
    } else {
        prev->next = current->next;
    }
    free(current);
}
```

Unsorted Singly Linked List

Find Min Operation

```
// Function to find the minimum value in the list
int findMin(Node* head) {
    if (head == NULL) {
        printf("List is empty\n");
        exit(1);
    }
    int min = head->data;
    Node* current = head->next;
    while (current != NULL) {
        if (current->data < min) {
            min = current->data;
        }
        current = current->next;
    }
    return min;
}
```

Unsorted Singly Linked List

Delete Min Operation

```
// Function to delete the node containing the minimum value
void deleteMin(Node** headRef) {
    int min = findMin(*headRef);
    deleteByKey(headRef, min);
}
```

Unsorted Singly Linked List

Decrease Key Operation

```
// Function to decrease the key of a node
void decreaseKey(Node* node, int newKey) {
    if (node == NULL) {
        printf("Node not found\n");
        return;
    }
    if (node->data > newKey)
        node->data = newKey;
}
```

Unsorted Singly Linked List

Meld Operation

```
// Function to merge two linked lists (meld operation)
void meld(Node** head1, Node** head2) {
    if (*head1 == NULL) {
        *head1 = *head2;
    } else {
        Node* current = *head1;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = *head2;
    }
    *head2 = NULL; // Setting head2 to NULL to avoid dangling pointers
}
```

Sorted Singly Linked List

Sorted Singly Linked List

Insert Operation

```
// Function to insert a node into the sorted list
void insert(Node** headRef, int value) {
    Node* newNode = createNode(value);
    if (*headRef == NULL || (*headRef)->data >= value) {
        newNode->next = *headRef;
        *headRef = newNode;
    } else {
        Node* current = *headRef;
        while (current->next != NULL && current->next->data < value) {
            current = current->next;
        }
        newNode->next = current->next;
        current->next = newNode;
    }
}
```

Sorted Singly Linked List

Search, DeleteByPointer, DeleteByKey Operations

Unchanged from unsorted singly linked lists

Sorted Singly Linked List

Find Min Operation

```
// Function to find the minimum value in the list
int findMin(Node* head) {
    if (head == NULL) {
        printf("List is empty\n");
        exit(1);
    }
    return head->data;
}
```

Sorted Singly Linked List

Delete Min Operation

```
// Function to delete the node containing the minimum value
void deleteMin(Node** headRef) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        exit(1);
    }
    deleteByPtr(headRef, *headRef);
}
```

Sorted Singly Linked List

Decrease Key Operation

```
// Function to decrease the key of a node
void decreaseKey(Node** headRef, Node* node, int newKey) {
    if (node == NULL) {
        printf("Node not found\n");
        return;
    }
    if (node->data > newKey) {
        DeleteByPtr(headRef, node);
        insert(headRef, newKey);
    }
}
```

Unsorted Doubly Linked List

Unsorted Doubly Linked List

Introduction

```
// Node structure for the linked list
typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

// Function to create a new node
Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}
```

Unsorted Doubly Linked List

Operations: Overview

- ▶ **Search:** Linear search through the list is all that can be done.
- ▶ **Insert:** Insert at the head of the list.
- ▶ **Delete (specified with pointer):** Make the previous and next nodes point to each other.
- ▶ **Delete (specified with key):** Locate the node. Make the previous and next nodes point to each other.
- ▶ **FindMin:** Linear search through the list is all that can be done.
- ▶ **DeleteMin:** Locate the node. Make the previous and next nodes point to each other.
- ▶ **DecreaseKey:** Decrease the key of the pointed node.
- ▶ **Meld:** Locate the last node of the first list, make it the predecessor of the head of the other.

Unsorted Doubly Linked List

Time bounds

If n is the current size of the data structure,

- ▶ **Search:** $O(n)$
- ▶ **Insert:** $O(1)$
- ▶ **Delete (specified with pointer):** $O(1)$.
- ▶ **Delete (specified with key):** $O(n)$.
- ▶ **FindMin:** $O(n)$.
- ▶ **DeleteMin:** $O(n)$.
- ▶ **DecreaseKey:** $O(1)$.
- ▶ **Meld:** $O(n)$. ($O(1)$ if a pointer to the last node is available.)